

El Fantasma de la máquina

- **Límite de tiempo:** 10 minutos
- **Puntuación del baseline:** 28.6
- **Entorno:** una GPU (≈16 GB de VRAM), sin internet
- **Tamaño de la solución:** `solution.ipynb` ≤ 20 MB
- **Almacenamiento:** 5 GB
- **Modelos preentrenados:** únicamente [bge-base-en-v1.5](#), un **codificador** de texto (modelo de embeddings).

Tarea

Están sucediendo cosas extrañas en el Archivo Nacional de Kazajistán. Los bibliotecarios afirman que algunos libros antes terminaban de manera diferente, pero nadie puede demostrarlo: todas las copias son iguales y todas las historias siguen teniendo sentido. Se le invita, en calidad de investigador de IA, a localizar los cambios.



Un pasaje comienza como texto escrito por una persona y, en algún momento, cambia silenciosamente a una continuación generada por un modelo de lenguaje. Leído en su conjunto, parece una única pieza coherente, pero en algún punto intermedio el autor cambia de una persona a una máquina. Su tarea consiste en **encontrar ese cambio: el índice de carácter donde termina la parte humana y comienza la parte de la máquina**.

Cada muestra es una única cadena `text`. Hay exactamente un solo límite de carácter. Todo lo anterior a este límite es humano; todo lo que se encuentra a partir de este ha sido generado por una máquina.

Dataset

Pasajes en inglés de texto plano, cada uno con un límite.

- **Parte A** (antes del límite): un fragmento de texto escrito por una persona.
- **Parte B** (a partir del límite): una continuación producida por un modelo de lenguaje, condicionada por la Parte A.
- Cada parte tiene al menos 180 palabras; la longitud total es entre 500 y 800 palabras.
- El `boundary_char_index` es la posición del carácter donde *comienza* la Parte B: `text[:boundary_char_index]` es la parte humana y `text[boundary_char_index:].rstrip()` es la parte de la máquina.

Material proporcionado

Recibirá **dos carpetas**:

Carpeta	Muestras	¿answers.jsonl?	Utilícela para
dataset/train/	1,221	❑ incluido	entrenar / realizar fine-tuning de su método
dataset/test_public/	380	❑ incluido (copia de desarrollo)	ejecutar su programa y autoevaluarlo localmente

En el **momento de la evaluación**, su carpeta `dataset/test_public/` se **sustituye automáticamente por un conjunto de evaluación oculto**. Tiene el mismo formato, pero **sin el archivo `answers.jsonl`**. Su notebook se vuelve a ejecutar sobre este, y se puntúa el archivo `answers.jsonl` que produce.

- La tabla de clasificación pública utiliza un conjunto **test_leaderboard_a** oculto con 380 muestras.
- La clasificación final utiliza un conjunto oculto **test_leaderboard_b** con 380 muestras.

Los tres conjuntos de evaluación tienen el mismo tamaño y se extraen de la misma distribución que `train`, por lo que su puntuación local de `dataset/test_public/` es una estimación razonable de su puntuación en la tabla de clasificación final.

Formato de directorios

```
dataset/train/data.jsonl      # un objeto en formato JSON por línea: {"id": "...", "text": "..."}
dataset/train/answers.jsonl  # {"id": "...", "boundary_char_index": numero}
dataset/test_public/data.jsonl    # {"id": "...", "text": "..."}
dataset/test_public/answers.jsonl # presente en la copia de desarrollo. AUSENTE en el dataset oculto
```

- Los IDs de `answers.jsonl` coinciden con los IDs de `data.jsonl`.
- `dataset/train/` (con respuestas) estará disponible siempre, ya sea durante el entrenamiento o cuando realice fine-tuning.

Salida (formato de entrega)

Debe entregar **un único notebook, que debe llamarse `solution.ipynb`**. Se exige este nombre de archivo exacto. Cualquier otro nombre de archivo será rechazado sin ejecutarse.

Su notebook debe leer `dataset/test_public/data.jsonl` y escribir un único archivo `answers.jsonl` en la raíz del repositorio: un objeto JSON por línea, que asocie cada ID de muestra con su predicción del índice de límite de carácter:

```
{"id": "example_000123", "boundary_char_index": 1790}
{"id": "example_000124", "boundary_char_index": 2450}
```

- `boundary_char_index` debe ser un **entero en el rango `[0, len(text)]`**.
- Cada ID de `dataset/test_public/data.jsonl` debe aparecer exactamente una vez. Una muestra ausente de `answers.jsonl` (o con un valor no entero / fuera de rango) obtiene 0 puntos para esa muestra.

Puntuación

Para cada muestra, sea p el índice predicho y t el límite verdadero. La puntuación por muestra decrece exponencialmente con la distancia entre caracteres:

$$\text{score} = \exp\left(-\frac{|p-t|}{\tau}\right) \in (0, 1], \text{ where } \tau = 100.$$

Esto da lugar al siguiente comportamiento de la puntuación:

- **≈1.0** — carácter de límite exacto;
- **≈0.78** — hasta 25 caracteres;
- **≈0.61** — hasta 50 caracteres;
- **≈0.37** — hasta 100 caracteres;
- **≈0.01** — hasta 500 caracteres.

La **puntuación final es la media** de las puntuaciones por muestra de todas las muestras de la partición (expresada en una escala de 0–100). La métrica recompensa las aproximaciones, no solo acertar exactamente.

Restricciones

- **Entorno:** una GPU (≈16 GB de VRAM), sin internet en el momento de la evaluación; el modelo permitido (indicado a continuación) ya está disponible. **Tiempo real disponible: 10 minutos** para la ejecución completa; esto debe abarcar cualquier entrenamiento / fine-tuning que realice en el momento de la evaluación, **además de** las inferencias sobre el conjunto de evaluación.
- **Modelo preentrenado permitido:** esta lista es exhaustiva; no se puede utilizar ningún otro conjunto de pesos preentrenados. Está **disponible previamente en el entorno** (cárguelo de la forma habitual, p. ej., `from_pretrained`; no hay internet en el momento de la evaluación):
 - **bge-base-en-v1.5:** un **codificador** de texto de 110M parámetros (modelo de embeddings). Este produce embeddings de oraciones/pasajes; no es un modelo de lenguaje generativo. Puede utilizarlo **tal como viene (características predefinidas) o realizar fine-tuning sobre la partición `train`**. El fine-tuning completo debe ajustarse al límite de 16 GB / 10 minutos.
- Las herramientas clásicas / estadísticas no están restringidas: puede construir cualquier modelo basado en características (p. ej., clasificadores o regresores de `scikit-learn`) sobre las características de embeddings que calcule usted mismo. Los *pesos preentrenados de deep learning* están restringidos únicamente a la lista anterior.